МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
«Санкт-Петербургский политехнический университет Петра Великого»
Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта
Направление: 02.03.01. «Математика и компьютерные науки»

Отчет о выполнении научно-исследовательской работы На тему: «Разработка имитатора пользовательских заявок и анализ эффективности распределения вычислительных ресурсов в планировщике SLURM»

Выполнил студент
группы 5130201/40002 _____ Храмов В.Д.

Принял старший преподаватель _____ Чуватов М.В.

«___» _____ 2026 г.

Санкт-Петербург
2026

РЕФЕРАТ

Отчёт о научно-исследовательской работе изложен на 31 странице, содержит 3 иллюстрации, 5 таблиц, 9 использованных источников, 1 приложение.

Ключевые слова: распределённые вычислительные системы, диспетчер заданий, SLURM, имитационное моделирование, нормальное распределение, логнормальное распределение, анализ датасета, виртуальный кластер, SSH.

Объектом исследования являются методы диспетчеризации и планирования вычислительных ресурсов в распределённых кластерных системах под управлением системы управления заданиями SLURM на базе операционной системы Linux в среде виртуализации.

Целью работы является разработка программного имитатора пользовательских заявок на основе статистического анализа исторических данных вычислительного кластера (вариант: UID 50430, JobName run_bwa.sh) и исследование динамики прохождения сформированных заявок через реальный диспетчер SLURM с применением механизмов масштабирования времени.

Методология работы включала развёртывание виртуального вычислительного кластера из пяти узлов средствами Oracle VM VirtualBox, статистический анализ выборки учётных данных acct_0921-0923 с вычислением границ диапазона фактического времени выполнения и параметров логарифма ошибки оценки времени выполнения, формируемой пользователем, а также натурный эксперимент по постановке сгенерированных заявок в очередь диспетчера.

Результатом разработки стал комплекс программно-управляемых скриптов на языках bash и Python, обеспечивающий фильтрацию исходного датасета, генерацию случайного потока заявок с усечением границ диапазона и их автоматическую отправку в пакетный диспетчер SLURM. Средствами SLURM собрана и проанализирована итоговая статистика соотношения «план — факт» для верификации использованной статистической модели.

Результаты работы могут быть использованы в учебном процессе при изучении технологий организации высокопроизводительных вычислений, администрирования распределённых систем под управлением SLURM.

Содержание

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ 4

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ 7

ВВЕДЕНИЕ 8

1 ОСНОВНАЯ ЧАСТЬ 10

1.1 Постановка задачи 10

1.2 Организация виртуальной вычислительной среды 12

1.3 Разработка имитатора пользовательских заявок 16

1.4 Результаты экспериментов и анализ данных 22

ЗАКЛЮЧЕНИЕ 25

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 27

ПРИЛОЖЕНИЕ А. ИСХОДНЫЙ КОД 28

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ

В настоящем отчете о НИР применяют следующие термины с соответствующими определениями:

1. Виртуализация — технология создания программного (виртуального) представления вычислительных ресурсов (процессора, памяти, накопителей, сетевых интерфейсов), позволяющая запускать несколько независимых операционных систем на одном физическом устройстве.
2. Виртуальная машина (ВМ) — программная эмуляция вычислительного устройства, предоставляющая гостевой операционной системе доступ к выделенным ей виртуальным аппаратным ресурсам посредством виртуализации.
3. VirtualBox (Oracle VM VirtualBox) — программный продукт виртуализации для операционных систем Windows, Linux, macOS и других, позволяющий запускать гостевые операционные системы в изолированной среде виртуальных машин на базе хост-системы.
4. Вычислительный кластер — совокупность взаимосвязанных вычислительных узлов, объединённых высокоскоростной сетью и функционирующих как единая вычислительная система под управлением специализированного программного обеспечения.
5. IP-адрес (Internet Protocol address) — уникальный числовой идентификатор сетевого интерфейса узла в компьютерной сети, используемый для адресации и маршрутизации пакетов данных по протоколу IP.
6. TCP-соединение (Transmission Control Protocol) — логическое соединение между двумя узлами сети, устанавливаемое по протоколу TCP и обеспечивающее надёжную, упорядоченную и контролируемую на предмет ошибок передачу данных между ними.
7. SSH (Secure Shell) — сетевой протокол прикладного уровня, позволяющий производить удалённое управление операционной системой и туннелирование TCP-соединений, в том числе с использованием авторизации по криптографическим ключам без передачи пароля по сети.
8. DNS (Domain Name System) — распределённая система преобразования символьных доменных имён узлов сети в соответствующие им

IP-адреса и обратно.

9. Разрешение имён — процесс сопоставления символьных имён узлов сети соответствующим им IP-адресам, выполняемый средствами DNS или локального файла соответствий (/etc/hosts).
10. Диспетчер заданий — программный компонент системы управления кластером, отвечающий за приём заявок от пользователей, ведение очереди заданий, планирование и распределение ресурсов вычислительных узлов.
11. Управляющий узел (head node, control node) — узел вычислительного кластера, на котором выполняется диспетчер заданий и средства управления кластером, отвечающие за приём, планирование и распределение заданий по вычислительным узлам.
12. Вычислительный узел (compute node) — узел вычислительного кластера, предоставляющий ресурсы (процессорное время, память) для непосредственного выполнения заданий, поставленных в очередь диспетчера.
13. Munge — служба аутентификации, используемая для проверки подлинности сообщений, которыми обмениваются узлы кластера, на основе общего секретного ключа.
14. SLURM (Simple Linux Utility for Resource Management) — система управления заданиями и ресурсами вычислительного кластера с открытым исходным кодом, обеспечивающая постановку заданий в очередь, их планирование и распределение по вычислительным узлам; для аутентификации узлов использует службу Munge.
15. Заявка (задание, job) — описание вычислительной задачи, поставленной пользователем в очередь диспетчера кластера, включающее запрошенные ресурсы и оценку времени выполнения.
16. Очередь заданий диспетчера — упорядоченный список заявок, принятых диспетчером SLURM и ожидающих выделения ресурсов для запуска на вычислительных узлах.
17. Среднее арифметическое — статистическая характеристика выборки, равная сумме всех её значений, делённой на их количество.
18. Дисперсия — статистическая характеристика разброса значений случайной величины, равная среднему арифметическому квадратов от-

клонений значений от среднего арифметического выборки.

19. Стандартное отклонение — статистическая характеристика разброса значений случайной величины относительно её среднего арифметического, равная квадратному корню из дисперсии выборки.
20. Нормальное распределение — непрерывное распределение вероятностей случайной величины, полностью определяемое математическим ожиданием и стандартным отклонением; график плотности имеет симметричную колоколообразную форму.
21. Логнормальное распределение — распределение вероятностей случайной величины, логарифм которой имеет нормальное распределение с параметрами μ и σ.

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

В настоящем отчете о НИР применяют следующие сокращения и обозначения:

ВМ — виртуальная машина

ОЗУ — оперативное запоминающее устройство

ОС — операционная система

ПО — программное обеспечение

ЦП — центральный процессор

CPU (Central Processing Unit) — центральный процессор

DNS (Domain Name System) — система доменных имён

GID (Group Identifier) — идентификатор группы пользователей

HPC (High Performance Computing) — высокопроизводительные вычисления

IP (Internet Protocol) — межсетевой протокол

Munge (MUNGE Uid 'N' Gid Emporium) — служба аутентификации узлов кластера на основе общего секретного ключа

NAT (Network Address Translation) — преобразование сетевых адресов

RAM (Random Access Memory) — оперативная память

SLURM (Simple Linux Utility for Resource Management) — система управления заданиями и ресурсами вычислительного кластера

SSH (Secure Shell) — сетевой протокол прикладного уровня для удалённого управления операционной системой

TCP (Transmission Control Protocol) — протокол управления передачей

UID (User Identifier) — идентификатор пользователя

ВВЕДЕНИЕ

В современном мире, где объёмы обрабатываемых данных постоянно растут, а вычислительные задачи становятся всё сложнее, технологии высокопроизводительных вычислений (HPC) [7] приобретают критически важное значение. Их применение позволяет объединять ресурсы множества вычислительных узлов в единый кластер, обеспечивая тем самым существенное увеличение производительности и сокращение времени решения сложных научных и инженерных задач.

Эффективная эксплуатация вычислительного кластера невозможна без системы управления заданиями и ресурсами, обеспечивающей приём заявок пользователей, их постановку в очередь и распределение по вычислительным узлам в соответствии с заданной политикой планирования. Одной из наиболее распространённых систем такого класса является SLURM [6; 8], широко используемая как в коммерческих, так и в научно-исследовательских вычислительных центрах.

Освоение принципов администрирования и эксплуатации вычислительного кластера на базе SLURM не требует обязательного наличия физического кластера: для учебных и исследовательских целей вычислительная среда может быть воспроизведена в виде набора виртуальных машин. Такие инструменты, как VirtualBox [4], позволяют создавать изолированные виртуальные среды, в которых каждая машина выполняет роль отдельного узла кластера, что делает возможным изучение архитектуры распределённой системы средствами одного физического компьютера.

Операционные системы семейства Linux, в частности дистрибутив Fedora Server [1], благодаря своей гибкости, открытости и широкому набору системных утилит, предоставляют все необходимые инструменты для организации управляющего и вычислительных узлов кластера в режиме интерфейса командной строки. Для организации безопасного взаимодействия между узлами кластера и удалённого администрирования незаменим протокол SSH, обеспечивающий авторизацию по криптографическим ключам без передачи пароля по сети, а также служба munge, отвечающая за аутентификацию сообщений между демонами SLURM.

Помимо развёртывания и настройки самой вычислительной среды, важной задачей является имитация реальной пользовательской нагрузки

на кластер с целью проверки корректности работы диспетчера и последующего анализа эффективности планирования заданий. Для построения такого имитатора целесообразно опираться не на произвольные, а на статистически обоснованные параметры заявок, полученные путём анализа реальных данных учёта выполненных задач действующего вычислительного кластера. Применение математического аппарата теории вероятностей и математической статистики, в частности нормального и логнормального распределений [9], позволяет генерировать синтетические заявки, по своим временным характеристикам приближенные к наблюдаемым в реальных системах.

В ходе настоящей НИР рассматриваются технологии организации вычислительного кластера на базе виртуализации и Linux, включая развёртывание управляющего и вычислительных узлов средствами VirtualBox, настройку сетевого взаимодействия и SSH-доступа, установку и конфигурирование диспетчера SLURM, а также разработку имитатора пользовательских заявок, основанного на статистическом анализе набора учётных данных кластера.

1 ОСНОВНАЯ ЧАСТЬ

1.1 Постановка задачи

В соответствии с выданным заданием (вариант: UID - 50430, JobName - run_bwa.sh) в рамках научно-исследовательской работы необходимо решить комплекс задач, разделённый на три основных этапа:

1. Подготовка виртуальной среды кластера. Необходимо развернуть и настроить виртуальную среду, имитирующую инфраструктуру распределённых вычислительных ресурсов суперкомпьютера, со следующими техническими характеристиками и требованиями:
   (a) Развернуть 5 виртуальных машин в среде виртуализации Oracle VM VirtualBox: один управляющий узел (master) с диспетчером ресурсов SLURM и средствами разработки, а также 4 вычислительных узла (node1–node4).
   (b) Выделить под каждый узел конфигурацию в пределах 1–2 виртуальных ЦП и 2–4 ГБ оперативной памяти. На всех машинах должна функционировать операционная система Linux в режиме интерфейса командной строки (CLI) без графической оболочки.
   (c) Настроить изолированную виртуальную сеть между всеми узлами с поддержкой разрешения имён.
   (d) Обеспечить беспарольный доступ по протоколу SSH между управляющим и вычислительными узлами с авторизацией по ключам.
   (e) Выполнить установку, настройку и запуск распределённого пакетного диспетчера ресурсов SLURM. Ресурсной единицей планирования определить вычислительный узел; другие типы ресурсов (ядра, оперативная память, дисковые квоты) в рамках данной работы не учитываются.
   (f) Все действия в рамках экспериментов должны выполняться под единой для всех узлов непривилегированной учётной записью пользователя (vlad). Суперпользователь root применяется исключительно для конфигурирования системного ПО.
2. Разработка имитатора пользовательских заявок. Программный ком-

плекс имитации должен реализовывать статистическую модель генерации нагрузки на основе исторических данных и включать в себя:

(a) Предварительный статистический анализ исторического набора данных acct_0921-0923. Требуется отфильтровать записи, соответствующие индивидуальному варианту, и вычислить:
- Границы диапазона фактического времени выполнения задач ($T_{\min}$ и $T_{\max}$);
- Параметры логнормального распределения ошибки оценки времени: среднее арифметическое логарифмов ошибки (μ) и их стандартное отклонение (σ).

(b) Реализацию фазы масштабирования времени с применением понижающих коэффициентов к временным интервалам ($T_{\min}, T_{\max}$) для возможности проведения натурного эксперимента в разумные сроки без искажения математической логики модели.

(c) Использование относительной величины ошибки оценки времени $\tau = T_{\text{заявки}}/T_{\text{факт}}$ (ввиду соотношения $\Delta T > T_{\text{факт}}$ для текущего варианта) для расчёта планового времени.

(d) Алгоритм генерации отдельной заявки, где:
- Фактическое время выполнения (t_{fact}) моделируется как случайная величина в усечённом диапазоне $[T_{\min}, T_{\max}]$ по нормальному закону распределения;
- Запрашиваемое пользователем время (t_{req}) вычисляется на основе логнормального отклонения τ от сгенерированного значения t_{fact};
- Потребность в вычислительных узлах выбирается из дискретного множества $\{1, 2, 3, 4\}$ на основе равномерного распределения.

(e) Контроль входных параметров: имитатор должен принимать на вход объём выборки N (целое положительное число, $1 \leq N \leq 999$).

(f) Автоматическую и последовательную отправку сформированных заявок в реальную очередь диспетчера SLURM.

3. Сбор статистики и верификация эксперимента. Для оценки адекватности работы имитатора необходимо организовать систему объектив-

ного контроля:

(a) Реализовать автоматизированный сбор данных непосредственно в процессе прохождения задач через диспетчер ресурсов (метрика «план – факт»).

(b) Сохранять в независимые файлы логов как исходные сгенерированные параметры («план» имитатора), так и фактические системные метрики времени выполнения и терминальных состояний процессов, зафиксированные средствами SLURM («факт» планировщика).

1.2 Организация виртуальной вычислительной среды

Выбор и развёртывание среды виртуализации

В качестве среды виртуализации в данной работе использовался программный продукт Oracle VM VirtualBox [4], выбор которого обусловлен его свободным распространением, поддержкой широкого спектра гостевых операционных систем и наличием официальной сборки для архитектуры ARM64, что является необходимым условием для развёртывания виртуальной среды на хост-системе с процессором Apple Silicon (MacBook Air, чип Apple M2, 8 ГБ ОЗУ).

В соответствии с вариантом задания в качестве гостевой операционной системы был выбран дистрибутив Fedora Server [1] в редакции для архитектуры aarch64. Установка и функционирование системы осуществлялись в режиме интерфейса командной строки, без графической оболочки.

Архитектура виртуального кластера

Для имитации вычислительных ресурсов суперкомпьютера было развёрнуто пять виртуальных машин: один управляющий узел (master), на котором размещаются диспетчер SLURM и средства разработки, и четыре вычислительных узла (node1-node4), предоставляющие ресурсы для выполнения заданий. Конфигурация виртуальных машин, выбранная с учётом ограничений хост-системы по объёму оперативной памяти, приведена в таблице 1.

Таблица 1: Конфигурация узлов виртуального кластера

Узел	Роль	ЦП (ядра)	ОЗУ (МБ)
master	управляющий узел	2	2048
node1	вычислительный узел	1	1024
node2	вычислительный узел	1	1024
node3	вычислительный узел	1	1024
node4	вычислительный узел	1	1024

На каждом узле создана единая непривилегированная учётная запись пользователя(vlad), используемая для запуска заданий и работы с диспетчером SLURM; учётная запись root применялась исключительно для установки и настройки системного программного обеспечения.

Настройка виртуальной сети и разрешения имён

Для обеспечения сетевого взаимодействия внутри кластера на каждой виртуальной машине был настроен сетевой адаптер в режиме внутренней сети (Internal Network), формирующий изолированную локальную подсеть 192.168.1.0/24. Для обеспечения доступа с хост-машины к управляющему узлу дополнительно задействован интерфейс NAT с пробросом порта 2222 хоста на порт 22 гостевой ОС. Каждому узлу присвоен статический IP-адрес согласно таблице 2.

Таблица 2: Адресация узлов виртуального кластера

Имя узла	IP-адрес
master	192.168.1.10
node1	192.168.1.11
node2	192.168.1.12
node3	192.168.1.13
node4	192.168.1.14

Разрешение имён узлов реализовано посредством редактирования файла /etc/hosts на каждой виртуальной машине. Данный подход позволяет обращаться к узлам по символьным именам (master, node1–node4) без развёртывания внешнего DNS-сервера и обеспечивает корректную работу демонов планировщика SLURM [6]. Итоговое содержимое файла приведено в листинге 1.

```
1 127.0.0.1     localhost
```

```
192.168.1.10 master
192.168.1.11 node1
192.168.1.12 node2
192.168.1.13 node3
192.168.1.14 node4
```

Листинг 1: Содержимое файла /etc/hosts

Настройка SSH с авторизацией по ключам

На управляющем узле для непривилегированной учётной записи сгенерирована пара криптографических ключей — приватный и публичный — с использованием утилиты ssh-keygen (листинг 2). Публичный ключ распространён на все вычислительные узлы посредством утилиты ssh-copy-id, что обеспечивает выполнение удалённых подключений и команд без запроса пароля.

```
ssh-keygen -t ed25519 -C "cluster-key"
ssh-copy-id vlad@node1
ssh-copy-id vlad@node2
ssh-copy-id vlad@node3
ssh-copy-id vlad@node4
```

Листинг 2: Генерация и распространение ключей SSH

В конфигурационном файле демона SSH (/etc/ssh/sshd_config) на всех узлах отключена авторизация по паролю посредством установки параметра PasswordAuthentication no, что исключает возможность входа без криптографического ключа.

Установка и настройка munge

Для аутентификации сообщений, которыми обмениваются демоны SLURM на разных узлах, на управляющем узле был сгенерирован общий секретный ключ munge.key, который был скопирован на все вычислительные узлы с сохранением единых владельца и прав доступа (пользователь munge). Служба munge была запущена и добавлена в автозагрузку на всех пяти узлах кластера.

Установка и настройка SLURM

На управляющем узле был установлен и настроен демон диспетчера slurmctld, на вычислительных узлах — демон slurmd. Основные параметры кластера определены в общем для всех узлов конфигурационном файле /etc/slurm/slurm.conf: имя кластера, адрес управляющего узла, список вычислительных узлов с указанием количества процессоров и объёма оперативной памяти, а также определение единственного раздела (partition), включающего все четыре вычислительных узла.

```
ClusterName=mac_cluster
SlurmctldHost=master
SlurmUser=slurm
SlurmdUser=root
AuthType=auth/munge
CryptoType=crypto/munge
SelectType=select/cons_tres
SelectTypeParameters=CR_Core
SlurmdParameters=config_overrides

# Логи
SlurmctldLogFile=/var/log/slurm/slurmctld.log
SlurmdLogFile=/var/log/slurm/slurmd.log
StateSaveLocation=/var/spool/slurmctld

# Описание машин
NodeName=node[1-4] CPUs=1 State=UNKNOWN
PartitionName=debug Nodes=node[1-4] Default=YES MaxTime=INFINITE State=UP
```

Листинг 3: Фрагмент файла slurm.conf

После запуска демонов на всех узлах корректность конфигурации и доступность вычислительных узлов проверялась командами sinfo и scontrol show nodes.

```
vlad@master:~$ sinfo
PARTITION AVAIL  TIMELIMIT  NODES  STATE NODELIST
debug*       up   infinite      4   idle node[1-4]
```

Листинг 4: Вывод команды sinfo

```
vlad@master:~$ scontrol show nodes
NodeName=node1 Arch=aarch64 CoresPerSocket=1
   CPUAlloc=0 CPUEfctv=1 CPUTot=1 CPULoad=0.18
   AvailableFeatures=(null)
```

```
   ActiveFeatures=(null)
   Gres=(null)
   NodeAddr=node1 NodeHostName=node1 Version=24.05.2
   OS=Linux 6.19.10-300.fc44.aarch64 #1 SMP PREEMPT_DYNAMIC Wed Mar 25
    17:45:07 UTC 2026
   RealMemory=1 AllocMem=0 FreeMem=463 Sockets=1 Boards=1
   State=IDLE ThreadsPerCore=1 TmpDisk=0 Weight=1 Owner=N/A MCS_label=N/A
   Partitions=debug
   BootTime=2026-06-25T06:44:34 SlurmdStartTime=2026-06-25T06:44:45
   LastBusyTime=2026-06-25T06:44:44 ResumeAfterTime=None
   CfgTRES=cpu=1,mem=1M,billing=1
   AllocTRES=
   CurrentWatts=0 AveWatts=0
```

Листинг 5: Фрагмент вывода команды scontrol show nodes

Клонирование вычислительных узлов

После полной настройки и проверки работоспособности первого вычислительного узла (установка ОС, сеть, SSH, munge, slurmd) данная виртуальная машина использовалась как базовый образ для клонирования оставшихся четырех вычислительных узлов средствами VirtualBox.

1.3 Разработка имитатора пользовательских заявок

Анализ исходного набора данных

Исходные данные для построения статистической модели пользовательских заявок представлены в виде набора учётных записей SLURM acct_0921-0923, содержащего сведения о реально выполненных задачах действующего вычислительного кластера. В соответствии с вариантом задания из общего набора данных были отобраны записи, удовлетворяющие условию: UID = 50430 и JobName = "run_bwa.sh".

В результате фильтрации было получено $N_{\text{sample}} = 1433$ записи, использованных для дальнейшего статистического анализа.

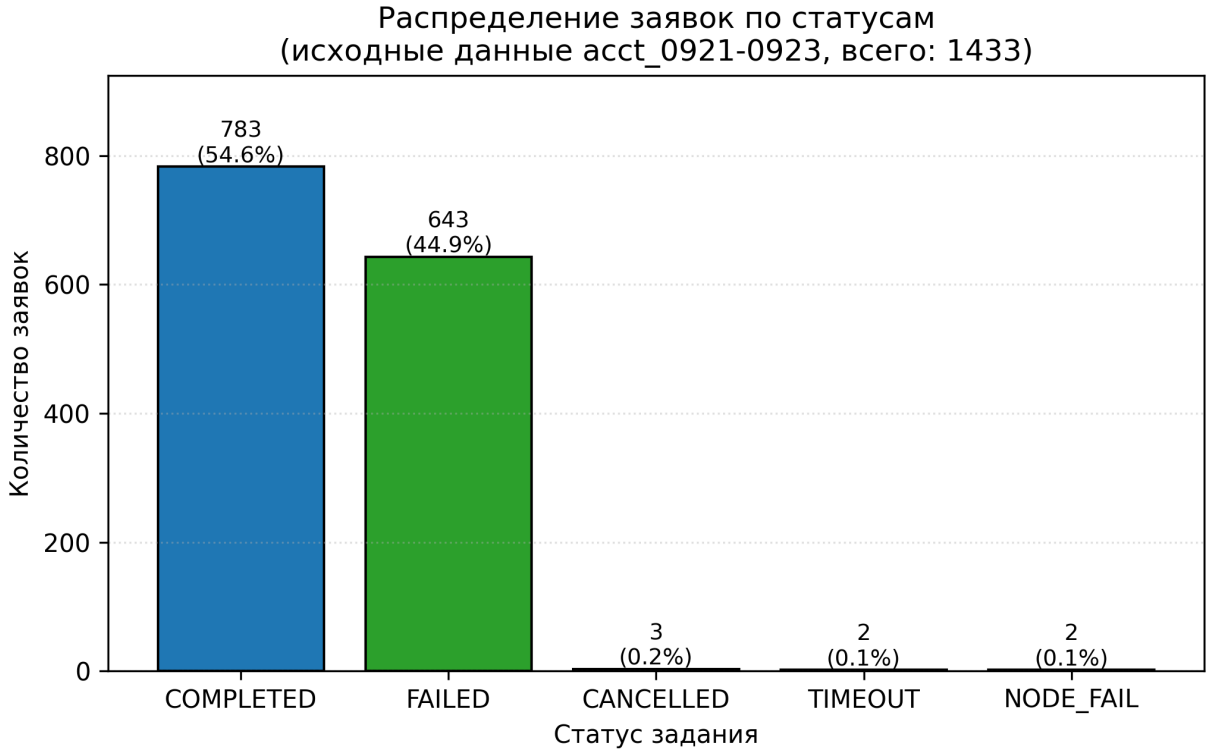


Рис. 1: Распределение заявок по статусам (исходные данные)

Предварительный анализ и масштабирование данных

Анализ отобранного подмножества данных показал, что фактическое время выполнения пользовательских задач (ElapsedRaw) варьируется в диапазоне от 26 секунд до 43209 секунд (что составляет более 12 часов). Статистический анализ логарифмированных значений интервалов времени ($\ln \tau$) позволил определить ключевые параметры распределения человеческой ошибки: математическое ожидание $\mu = 3.8621$ и среднеквадратичное отклонение $\sigma = 0.8489$.

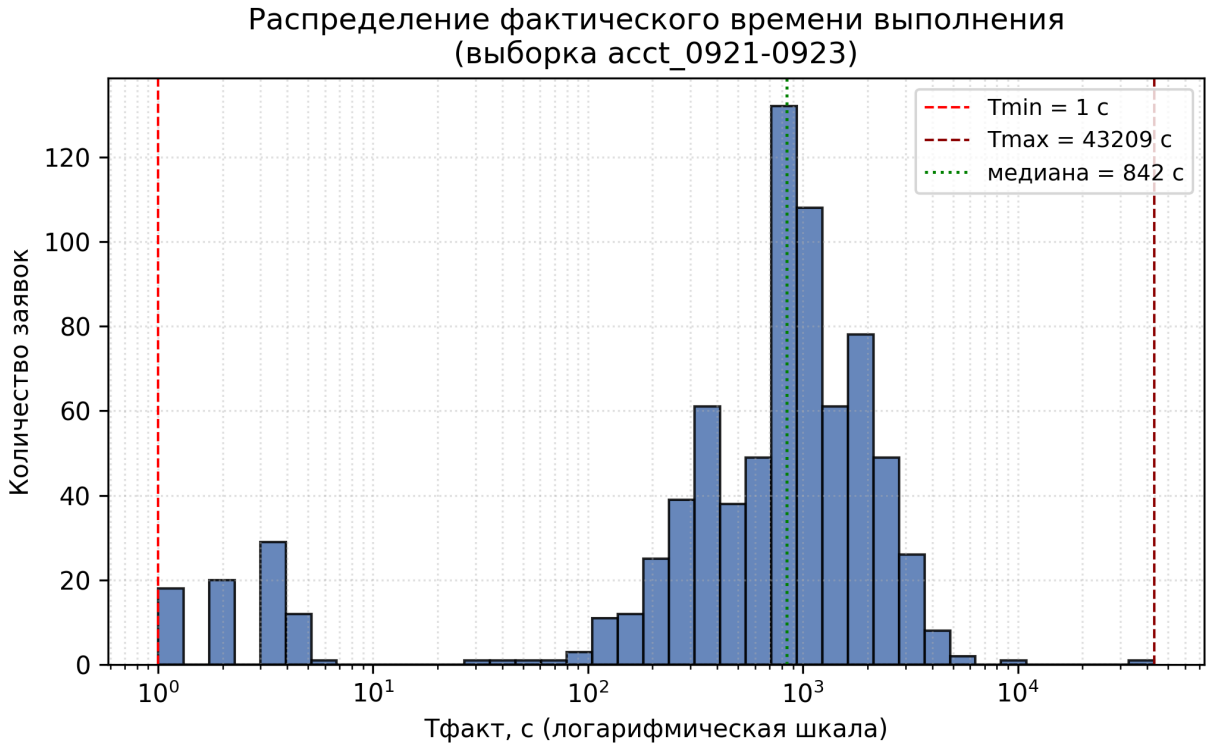


Рис. 2: Распределение фактического времени выполнения

Такой широкий временной диапазон существенно затрудняет непосредственное воспроизведение реальных интервалов выполнения задач в условиях учебного стенда. В связи с этим было принято решение о масштабировании исходных временных характеристик с понижающим коэффициентом $k = 1000$. Данное преобразование позволило перейти от исходных величин, измеряемых часами, к сжатым интервалам порядка единиц и десятков секунд (максимальное масштабированное время выполнения задачи составило всего 43.2 секунды). Такой масштаб оптимален для оперативной и наглядной практической демонстрации динамики заполнения очередей диспетчера SLURM в реальном времени непосредственно в процессе защиты работы.

Выбор модели ошибки оценки времени выполнения

Для характеристики отклонения пользовательской оценки времени [3] выполнения от фактического времени выполнения задачи использовались две альтернативные модели:

- абсолютное отклонение $\Delta T = T_{\text{заявки}} - T_{\text{факт}}$;
- относительное отклонение $\tau = T_{\text{заявки}}/T_{\text{факт}}$.

По результатам анализа отобранной выборки было установлено, что величина ΔT превышает величину $T_{\text{факт}}$, в связи с чем для дальнейшей работы была выбрана модель относительного отклонения τ.

Вычисление статистических параметров

Для отобранной и масштабированной выборки были вычислены границы диапазона фактического времени выполнения, а также параметры логарифма выбранной величины ошибки оценки. Результаты вычислений приведены в таблице 3.

Таблица 3: Статистические параметры выборки

Параметр	Значение
$T_{\text{факт, min}}$, с	26
$T_{\text{факт, max}}$, с	43209
$\mu\ (\ln \tau)$	3,8621
$\sigma\ (\ln \tau)$	0,8489

Таблица 4: Масштабированные параметры выборки ($k = 1000$)

Параметр	Исходное значение, с	Масштабированное значение, с
$T_{\text{факт, min}}$	26	$0{,}026 \approx 1$
$T_{\text{факт, max}}$	43209	$43{,}2 \approx 43$
$\mu\ (\ln \tau) = 3{,}8621$		
$\sigma\ (\ln \tau) = 0{,}8489$		

Алгоритм генерации пользовательской заявки

Генерация каждой синтетической заявки выполняется программно на языке Python [5] и состоит из нескольких ключевых этапов, базирующихся на статистических параметрах исходного датасета.

На первом этапе фактическое время выполнения задачи $T_{\text{факт}}$ моделируется как случайная величина, подчинённая нормальному распределению [9] $\mathcal{N}(\mu_t, \sigma_t^2)$. Математическое ожидание и стандартное отклонение задаются на основе масштабированных границ исследуемой выборки:

$$\mu_t = \frac{T_{\min} + T_{\max}}{2}, \qquad \sigma_t = \frac{T_{\max} - T_{\min}}{4}$$

Данный выбор параметров обеспечивает попадание более 95% генерируемых значений в требуемый диапазон $[T_{\min}, T_{\max}]$, где $T_{\min} = 1$ с, а $T_{\max} =$

43 с. Программная реализация осуществляет генерацию в цикле до тех пор, пока значение не удовлетворит критерию $T_{\min} \le T_{\text{факт}} \le T_{\max}$. Полученное время $T_{\text{факт}}$ округляется до целого числа секунд и используется в качестве параметра команды sleep в теле формируемого пакетного сценария.

На втором этапе пользовательская оценка времени выполнения задачи $T_{\text{заявки}}$ вычисляется как:

$$T_{\text{заявки}} = \text{round}(T_{\text{факт}} \cdot \tau), \qquad \tau \sim \mathcal{N}_{\log}(\mu, \sigma^2),$$

то есть отклонение оценки от фактического времени (величина человеческой ошибки τ) моделируется [2; 9] с вычисленными параметрами $\mu = 3.8621$ и $\sigma = 0.8489$. Программно заложена защита от некорректных значений: если сгенерированная оценка времени оказывается меньше или равна фактическому времени ($T_{\text{заявки}} \le T_{\text{факт}}$), лимит принудительно корректируется добавлением случайного интервала от 5 до 15 секунд. Это исключает преждевременное прерывание задачи демоном slurmctld.

Количество узлов, запрашиваемых заявкой, выбирается случайным образом из дискретного множества $\{1, 2, 3, 4\}$ с использованием равномерного распределения посредством встроенной функции random.choice(), независимо от количества узлов, фигурирующего в исходных данных по варианту.

При отправке задачи в очередь SLURM вычисленное значение $T_{\text{заявки}}$ в секундах транслируется в формат временного лимита диспетчера –time (MM:CC) с округлением в большую сторону до ближайшей минуты (минимум 1 минута), что гарантирует стабильность работы планировщика на коротких интервалах времени.

Реализация программы-имитатора

Имитатор реализован на языке Python 3 и принимает в качестве единственного аргумента командной строки количество генерируемых заявок N — целое число в диапазоне от 1 до 999. Перед началом генерации программа инициализирует рабочую директорию /home/vlad/saved_jobs, удаляя её содержимое при повторном запуске, и открывает файл статистики /home/vlad/simulation_stats.csv на запись с заголовком таблицы.

Для каждой из N заявок программа последовательно выполняет сле-

дующие действия.

Генерация фактического времени выполнения. Значение $T_{\text{факт}}$ вычисляется по усечённому нормальному распределению с математическим ожиданием $\mu_T = (T_{\min} + T_{\max})/2 = 22$ с и стандартным отклонением $\sigma_T = (T_{\max} - T_{\min})/4 = 10{,}5$ с в границах $[T_{\min}, T_{\max}] = [1, 43]$ с. Значение округляется до целого и ограничивается снизу двумя секундами.

Генерация запрашиваемого времени. С вероятностью $p_{\text{timeout}} = 0{,}05$ задача помечается как таймаутная: в этом случае лимит времени $T_{\text{заявки}}$ выбирается случайно из равномерного распределения на интервале $[1, T_{\text{факт}} - 1]$, заведомо меньше фактического. В противном случае вычисляется коэффициент перестраховки пользователя $\tau \sim \text{LogNormal}(\mu, \sigma)$ с параметрами $\mu = 3{,}8621$ и $\sigma = 0{,}8489$, после чего $T_{\text{заявки}} = \lceil T_{\text{факт}} \cdot \tau \rceil$. Если полученное значение не превышает $T_{\text{факт}}$, к нему добавляется случайная поправка от 5 до 15 секунд, гарантирующая выполнение условия $T_{\text{заявки}} > T_{\text{факт}}$.

Выбор числа вычислительных узлов. Количество запрашиваемых узлов вычисляется равномерно из множества $\{1, 2, 3, 4\}$.

Формирование и отправка пакетного сценария. На основе сгенерированных параметров программа формирует текстовый файл сценария в формате sbatch: указываются имя задачи run_bwa.sh, число узлов (–nodes), лимит времени в формате MM:CC (–time) и системная команда sleep $T_{\text{факт}}$ в качестве имитации вычислительной нагрузки. Сценарий передаётся планировщику через системный вызов утилиты sbatch [6], возвращаемый идентификатор задания SLURM (SlurmJobID) фиксируется программой.

Логирование и архивирование. Метаданные каждой заявки — временная метка, внутренний порядковый номер, SlurmJobID, число узлов, плановое и фактическое время выполнения, а также коэффициент τ — записываются в файл статистики в формате CSV. Файл сценария перемещается в директорию saved_jobs под именем, включающим оба идентификатора задания; в случае ошибки отправки файл сохраняется с суффиксом _ERROR.

Сбор статистики поставленных в очередь заявок

Для последующего анализа и сравнения теоретических (плановых) параметров с фактическими системными характеристиками выполненных

задач, сбор статистики осуществлялся параллельно двумя методами:

1. Программное логирование: скрипт-имитатор simulator.py в момент генерации фиксировал внутренние параметры распределений и расчетные значения в файл simulation_stats.csv.
2. Системный аудит SLURM: сбор метрик планировщика выполнялся утилитой sacct. Из базы данных подсистемы учета извлекались идентификатор задачи (JobID), имя (JobName), количество выделенных узлов (Nodes), финальное состояние (State) и точное астрономическое время выполнения (ElapsedRaw).

Результаты системного сбора экспортировались в текстовый файл slurm_real_fact.txt с использованием разделителя | для последующего сопоставления с логом имитатора.

Фрагменты полученных файлов данных, демонстрирующие успешное завершение первых пяти сгенерированных задач кластера в состоянии COMPLETED, представлены в листингах ниже.

```
Timestamp,SimID,SlurmJobID,Nodes,PlannedTime_Sec,ActualTime_Sec,Tau
2026-06-19 05:01:14,1,1294,1,6777,40,169.425
2026-06-19 05:01:14,2,1295,1,1074,14,76.7143
2026-06-19 05:01:14,3,1296,2,1352,36,37.5556
2026-06-19 05:01:14,4,1297,4,714,18,39.6667
2026-06-19 05:01:14,5,1298,1,306,22,13.9091
```

Листинг 6: Фрагмент файла статистики имитатора simulation_stats.csv

```
JobID|JobName|Nodes|State|ElapsedRaw
1294|run_bwa.sh|1|COMPLETED|41
1295|run_bwa.sh|1|COMPLETED|14
1296|run_bwa.sh|2|COMPLETED|36
1297|run_bwa.sh|4|COMPLETED|18
1298|run_bwa.sh|1|COMPLETED|22
```

Листинг 7: Фрагмент системного лога планировщика slurm_real_fact.txt

1.4 Результаты экспериментов и анализ данных

Условия проведения эксперимента

В рамках экспериментальной проверки работы имитатора было сгенерировано и последовательно поставлено в очередь диспетчера $N = 783$ синтетических заявок. Все четыре вычислительных узла кластера (node1–

node4) на момент начала эксперимента находились в состоянии полной готовности (idle), управляющий узел master успешно координировал распределение ресурсов.

Результаты постановки заявок в очередь

По завершении выполнения всех сгенерированных задач из базы данных подсистемы учёта SLURM и логов имитатора были собраны итоговые статистические характеристики. Сводные результаты сопоставления плановых параметров и фактических метрик выполнения для первых пяти заявок представлены в таблице 5.

Таблица 5: Пример сравнения плановых и фактических характеристик выполнения заявок

SlurmJobID	$T_{\text{заявки}}$ (план), с	$T_{\text{факт}}$, с	Узлов	Состояние
2657	1495	23	3	COMPLETED
2656	2210	34	1	COMPLETED
2658	1421	29	3	COMPLETED
2659	1127	23	2	COMPLETED
2660	1029	21	3	COMPLETED

Анализ отклонений «план – факт»

Сопоставление запрошенного пользователем времени выполнения $T_{\text{заявки}}$ и фактического времени выполнения $T_{\text{факт}}$ выполнено по результатам эксперимента с $N = 783$ заявками. Из общего числа заявок 47 (6,0 %) были поданы с лимитом времени, заведомо меньшим фактического, что соответствует теоретической вероятности таймаута $p_{\text{timeout}} = 0{,}05$. Для оставшихся 736 нормальных заявок по значениям $T_{\text{заявки}}$ и $T_{\text{факт}}$ вычислены выборочные логарифмы коэффициента $\tau = T_{\text{заявки}}/T_{\text{факт}}$, на основе которых получены эмпирические оценки параметров логнормального распределения [3; 7]: $\hat{\mu} = 3{,}8973$ и $\hat{\sigma} = 0{,}8578$.

Полученные значения демонстрируют высокую степень близости к теоретическим параметрам, использованным при генерации: $\mu = 3{,}8621$ и $\sigma = 0{,}8489$. Относительное отклонение по параметру μ составляет 0,9 %, по параметру σ — 1,1 %, что подтверждает статистическую адекватность разработанной модели при объёме выборки $N = 783$.

По данным файла учёта заданий SLURM, фактическое время выпол-

нения задач находится в диапазоне от 2 до 43 секунд при среднем значении 22,5 с и стандартном отклонении 9,4 с, что соответствует заданным границам модельного диапазона $[T_{\min}, T_{\max}] = [1, 43]$ с. Визуальное сопоставление эмпирических распределений коэффициента ошибки τ для исторических и сгенерированных данных представлено на рисунке 3.

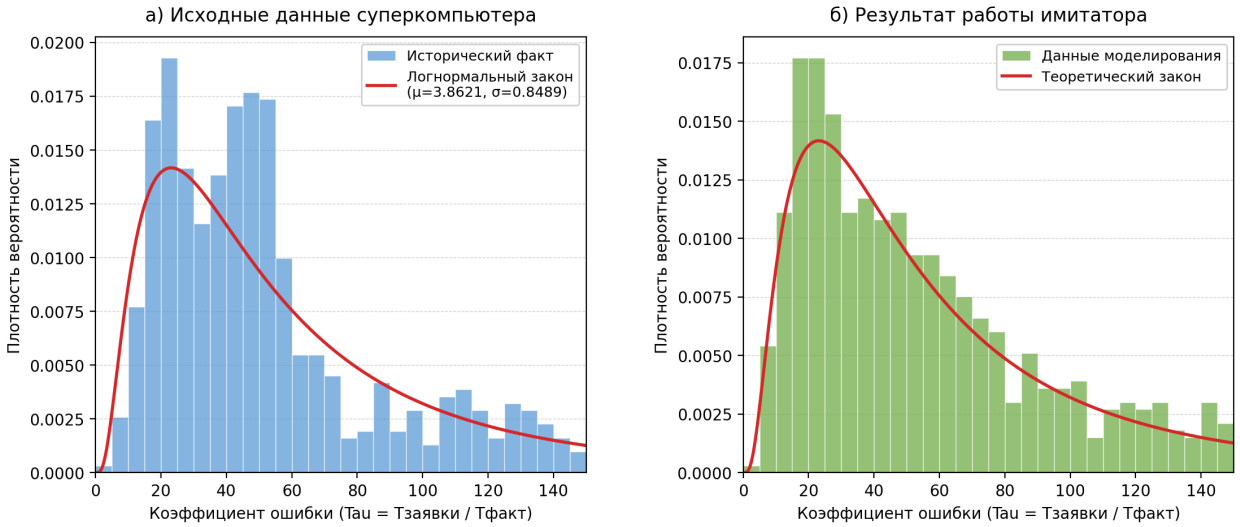


Рис. 3: Плотность распределения коэффициента ошибки пользователя τ: а) на основе исторических данных выборки acct_0921-0923; б) сгенерированная программным имитатором

Выводы по результатам экспериментов

Проведённый эксперимент показал, что разработанный имитатор пользовательских заявок на языке Python 3 обеспечивает корректную автоматическую постановку заданий в очередь диспетчера SLURM. Все 783 сгенерированные заявки были успешно приняты планировщиком и выполнены на вычислительных узлах кластера. Статистические характеристики сгенерированной нагрузки — эмпирические оценки параметров логнормального распределения $\hat{\mu} = 3{,}8973$ и $\hat{\sigma} = 0{,}8578$ — воспроизводят теоретические параметры исходной выборки с отклонением менее 1,2 %. Это подтверждает применимость предложенного статистического подхода к моделированию пользовательской нагрузки на вычислительный кластер при отсутствии доступа к данным реальной эксплуатационной системы в реальном масштабе времени.

ЗАКЛЮЧЕНИЕ

В процессе выполнения НИР были изучены принципы организации виртуальной вычислительной среды, имитирующей ресурсы вычислительного кластера, а также технологии управления заданиями средствами системы SLURM на базе дистрибутива Linux Fedora Server в среде виртуализации Oracle VM VirtualBox. В результате работы были выполнены поставленные задачи:

1. Установлена и настроена среда виртуализации Oracle VM VirtualBox.
2. Создано пять виртуальных машин: один управляющий узел и четыре вычислительных узла кластера.
3. Установлен дистрибутив Linux Fedora Server (архитектура aarch64) на все виртуальные машины в режиме интерфейса командной строки.
4. Настроена виртуальная локальная сеть между узлами кластера с разрешением имён посредством файла /etc/hosts.
5. Настроено подключение по протоколу SSH с авторизацией по криптографическим ключам между управляющим и вычислительными узлами.
6. Установлена и настроена служба аутентификации munge, а также диспетчер заданий SLURM.
7. Выполнено клонирование настроенного вычислительного узла для создания оставшихся узлов кластера с последующей индивидуализацией их сетевых параметров.
8. Проведён анализ набора учётных данных acct_0921-0923 по заданным атрибутам варианта (UID 50430, JobName “run_bwa.sh”), вычислены границы диапазона фактического времени выполнения задач, а также параметры μ и σ логарифма ошибки оценки времени выполнения.
9. Разработан имитатор пользовательских заявок, генерирующий фактическое время выполнения задачи по нормальному распределению в вычисленном диапазоне и пользовательскую оценку времени выполнения по логнормальному распределению с вычисленными параметрами, с автоматической постановкой сформированных заявок в очередь диспетчера SLURM.
10. Реализован сбор и сохранение в файл статистики поставленных в оче-

редь заявок для последующего сравнительного анализа плановых и фактических характеристик их выполнения.

Таким образом, в ходе выполнения НИР развёрнута работоспособная виртуальная модель вычислительного кластера, на базе которой реализован и экспериментально проверен имитатор пользовательской нагрузки, статистические характеристики которого согласуются с параметрами, полученными в результате анализа реального набора учётных данных. Полученные результаты могут быть использованы в качестве основы для дальнейшего расширения функциональности диспетчера и более детального анализа эффективности алгоритмов планирования заданий.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Fedora Project Contributors. Fedora Linux Documentation. — 2024. — Дата обращения: 25.06.2026. https://docs.fedoraproject.org/.
2. Job Failures in High Performance Computing Systems: A Large-scale Empirical Study / S. Di [и др.] // Computers & Mathematics with Applications. — 2014. — Т. 68, № 10. — С. 1224—1236. — DOI: 10.1016/j.camwa.2014.08.010.
3. Kappes T., Hanzálek Z. User Estimates Inaccuracy Study in HPC Scheduler // Parallel Processing and Applied Mathematics. Т. 10778. — Cham : Springer, 2018. — С. 486—495. — (Lecture Notes in Computer Science). — DOI: 10.1007/978-3-319-91446-6_47.
4. Oracle Corporation. Oracle VM VirtualBox User Manual, Release 7.2. — 2025. — Дата обращения: 25.06.2026. https://www.virtualbox.org/manual/.
5. Python Software Foundation. Python 3 Documentation. — 2024. — Дата обращения: 25.06.2026. https://docs.python.org/3/.
6. SchedMD LLC. Slurm Workload Manager — Documentation. — 2024. — Дата обращения: 25.06.2026. https://slurm.schedmd.com/documentation.html.
7. Sherrill C., Planas V., Golden J. Mastering HPC Runtime Prediction: From Observing Patterns to a Methodological Approach : тех. отч. / National Renewable Energy Laboratory. — Golden, CO, 2023. — NREL/TP-2C00—86526. — URL: https://docs.nrel.gov/docs/fy23osti/86526.pdf.
8. Yoo A. B., Jette M. A., Grondona M. SLURM: Simple Linux Utility for Resource Management // Job Scheduling Strategies for Parallel Processing. Т. 2862. — Berlin, Heidelberg : Springer, 2003. — С. 44—60. — (Lecture Notes in Computer Science). — DOI: 10.1007/10968987_3.
9. Гмурман В. Теория вероятностей и математическая статистика. — 10-е изд. — Москва : Высшая школа, 2003. — С. 479.

ПРИЛОЖЕНИЕ А. ИСХОДНЫЙ КОД

В приложении приведён полный исходный код программного имитатора пользовательских заявок (simulator.py), разработанного на языке Python 3. Программа выполняет статистический анализ набора учётных данных SLURM, генерирует синтетические заявки в соответствии с нормальным и логнормальным законами распределения и автоматически ставит их в очередь диспетчера посредством утилиты sbatch.

```
import os
import sys
import random
import math
import csv
import shutil
import subprocess
from datetime import datetime

# --- НАСТРОЙКИ ---
UID = 50430
JOB_NAME = "run_bwa.sh"

# --- МАСШТАБИРОВАННЫЕ ПАРАМЕТРЫ ИЗ ДАТАСЕТА ---
T_MIN = 1
T_MAX = 43

MU = 3.8621
SIGMA = 0.8489
TIMEOUT_RATE = 0.05

def generate_job():
    # 1. Фактическое время по усеченному нормальному распределению
    while True:
        mean_t = (T_MIN + T_MAX) / 2
        std_t = (T_MAX - T_MIN) / 4
        t_fact = random.normalvariate(mean_t, std_t)
        if T_MIN <= t_fact <= T_MAX:
            break

    t_fact = max(2, round(t_fact))

    # Определяем, будет ли эта задача таймаутом на основе исторической вероятности
    is_timeout = random.random() < TIMEOUT_RATE

    if is_timeout:
```

```
37          # Режим таймаута: лимит времени строго меньше фактического времени работы
38          t_req = random.randint(1, t_fact - 1)
39      else:
40          # Нормальный режим: логнормальная ошибка перестраховки пользователя
41          tau = random.lognormvariate(MU, SIGMA)
42          t_req = round(t_fact * tau)
43
44          # Гарантируем, что в нормальном режиме лимит всё-таки больше факта
45          if t_req <= t_fact:
46              t_req = t_fact + random.randint(5, 15)
47
48      # 3. Выбор количества узлов (Равномерное распределение)
49      nodes = random.choice([1, 2, 3, 4])
50
51      return t_fact, t_req, nodes
52
53  def submit_to_slurm(job_id, t_fact, t_req, nodes):
54      # Форматируем лимит времени в посекундный формат MM:CC
55      # Это заставит SLURM обрубать задачи по таймауту с точностью до секунды
56      mm = t_req // 60
57      ss = t_req % 60
58      timelimit_str = f"{mm:02d}:{ss:02d}"
59
60      sbatch_content = f"""#!/bin/bash
61  #SBATCH --job-name={JOB_NAME}
62  #SBATCH --nodes={nodes}
63  #SBATCH --time={timelimit_str}
64  #SBATCH --output=/dev/null
65  #SBATCH --error=/dev/null
66
67  sleep {t_fact}
68  """
69      # Папка, куда будут складываться все сгенерированные скрипты
70      jobs_dir = "/home/vlad/saved_jobs"
71      script_name = f"job_{job_id:02d}.sh"
72
73      with open(script_name, "w") as f:
74          f.write(sbatch_content)
75
76      try:
77          # Отправляем задачу в SLURM
78          result = subprocess.run(["sbatch", script_name], capture_output=True, text=True, check=True)
79          slurm_job_id = result.stdout.strip().split()[-1]
80          print(f"Заявка {job_id:02d} отправлена. ID: {slurm_job_id} | Лимит: {t_req:2d}c ({timelimit_str}) | Факт: {t_fact:2d}c | Будет TIMEOUT: {t_req < t_fact}")
```

```
81
82          # Переносим сохраненный скрипт в постоянную папку с указанием Slurm ID
83          final_script_name = f"job_{job_id:02d}_slurm_{slurm_job_id}.sh"
84          shutil.move(script_name, os.path.join(jobs_dir, final_script_name))
85
86          return slurm_job_id
87      except Exception as e:
88          print(f"Ошибка отправки в SLURM: {e}")
89          if os.path.exists(script_name):
90              shutil.move(script_name, os.path.join(jobs_dir, f"job_{job_id:02d}
    _ERROR.sh"))
91          return "ERROR"
92
93  def main():
94      if len(sys.argv) < 2:
95          print("Использование: python3 simulator.py <количество_заявок>")
96          sys.exit(1)
97
98      try:
99          num_jobs = int(sys.argv[1])
100         if not (1 <= num_jobs <= 999):
101             raise ValueError
102     except ValueError:
103         print("Ошибка: Количество заявок должно быть целым числом от 1 до 999.
    ")
104         sys.exit(1)
105
106     # Создаем/очищаем папку для скриптов задач перед началом нового эксперимен
    та
107     jobs_dir = "/home/vlad/saved_jobs"
108     if os.path.exists(jobs_dir):
109         shutil.rmtree(jobs_dir)
110     os.makedirs(jobs_dir, exist_ok=True)
111
112     log_file = "/home/vlad/simulation_stats.csv"
113
114     # Открываем файл в режиме "w" (перезапись для чистоты эксперимента)
115     with open(log_file, mode="w", newline="") as f:
116         writer = csv.writer(f)
117         writer.writerow(["Timestamp", "SimID", "SlurmJobID", "Nodes", "
    PlannedTime_Sec", "ActualTime_Sec", "Tau"])
118
119         print(f"--- Запуск генерации {num_jobs} заявок ---")
120         for i in range(1, num_jobs + 1):
121             t_fact, t_req, nodes = generate_job()
122             tau = round(t_req / t_fact, 4)
123
124             slurm_id = submit_to_slurm(i, t_fact, t_req, nodes)
```

```
125                 writer.writerow([datetime.now().strftime("%Y-%m-%d %H:%M:%S"), i,
    slurm_id, nodes, t_req, t_fact, tau])
126
127     print(f"--- Имитация завершена. Скрипты сохранены в {jobs_dir} ---")
128
129 if __name__ == "__main__":
130     main()
```

Листинг 8: Имитатор пользовательских заявок simulator.py